# Prefix complexity

Bookmark this page

**Question:** Why do we need a new version of Kolmogorov complexity?

**Answer:** Because "plain" Kolmogorov complexity (as defined in Chapter 1) is not sufficient to define algorithmic probability and randomness.

In a computer language such as Python, a program that generates $x$ can be extended by merely adding useless instructions (such as `pass`). As we will see, this possibility of extension makes $x$ "too" probable (in fact, probability defined on extensible programs is meaningless). One natural way to prevent program extension is to forbid it!

**Prefix Turing machines** are Turing machines with a restriction. No program interpreted by a Prefix machine $U$ is the continuation of another program that $U$ can interpret. This means that if a program $p$ can be interpreted by $U$, then $U$ must recognize that $p$ has come to an end and must unconditionally stop reading its input after reaching the end of $p$. This also means that if a program $q$ is the concatenation of two programs $p$ and $p_1$: $q = pp_1$, then $U$ does not read anything beyond the limit between $p$ and $p_1$.

---

## Prefix Machine 1

1/1 point (ungraded)

**Consider a Universal Turing Machine $U$ which halts for the following programs: `01000`, `01001`, `101` and `010`.**
**Can $U$ be a prefix machine?**

- ○ Yes
- ◉ No

✔

**Answer**
Correct: No, since `01001` is the continuation of `010`.

Submit **You have used 1 of 1 attempt**

**Answers are displayed within the problem**

---

## Prefix Machine 2

1/1 point (ungraded)

**If $U'$ halts for `1`, `01`, `001`, `0001`, `00001`, and for all programs of the form `0...01`, can $U'$ be a prefix machine?**

- ◉ Yes
- ○ No

✔

**Answer**
Correct:
Yes, U' can be a prefix machine, as none of these program is the continuation of another one.

Submit **You have used 1 of 1 attempt**

**Answers are displayed within the problem**

---

The restriction to prefix machines leads to a redefinition of complexity, introduced by Leonid Levin in 1974.
The *prefix complexity* of *s* is the size of the shortest program that outputs *s* when run on a given universal *prefix* Turing machine $U_{pr}$.

*Prefix complexity*

$$K(s) = \min_p \{|p| : U_{pr}(p) = s\}$$

By contrast, our previous definition of complexity, noted $C(s)$ (without the restriction to prefix machines) will be called ***plain* complexity**.

For any prefix machine $U_{pr}$, there is an ordinary Turing machine $U$ that can execute all of $U_{pr}$'s programs, plus many other (non prefix) programs. For instance, $U$ may use a systematic binary chunk to indicate that what comes next is a program in prefix form.

---

## K and C

1/1 point (ungraded)

**How do plain complexity and prefix complexity compare?**
**(here, $s$ is any object and $c$ is some constant that does not depend on $s$)**

- ○ $K(s) \leq C(s) + c$
- ○ it depends
- ◉ $C(s) \leq K(s) + c$

✔

**Answer**
Correct:
Taking $U$ as reference machine for $C$, we are sure that $C(s) < K(s)$, since $C(s)$ takes a minimum over more programs.
Note that prefix programs are handicapped by the fact that they must incorporate an indication about their length. Their non-prefix counterpart might be shorter.

Submit  You have used 1 of 1 attempt

✔ Correct (1/1 point)

‹ Previous  Next ›

## Legal

Terms of Service & Honor Code
Privacy Policy
Accessibility Policy
Trademark Policy
Sitemap
Cookie Policy
Your Privacy Choices

## Connect

Idea Hub
Contact Us
Help Center
Security
Media Kit